

DataPulse

Scraping web · APIs REST · Reporting automatisé

Collecter des données depuis le web et des APIs publiques :
scrapper une page HTML, interroger des APIs sans clé,
et automatiser la génération d'un rapport marketing complet.

ESD Lyon - M1 Data

Module : Workshop Programmation — Analyse de données

Pierre LACOGNATA (pierre@lacognata.io)

Ressources : [par ici](#)

Année scolaire : 2025 – 2026



lacognata.io

requests

BeautifulSoup

Open-Meteo

REST Countries

Frankfurter

pandas

PostgreSQL

SQLAlchemy





Table des matières

01	Contexte du projet — DataPulse et NovaBrand	3
	1.1 Ce que l'on va construire ·	
	1.2 Le web comme source de données ·	
	1.3 Les APIs : pourquoi et comment	
02	 Scraping web — requests et BeautifulSoup	4-7
	2.1 Comment fonctionne une page web ·	
	2.2 requests — récupérer du HTML ·	
	2.3 BeautifulSoup — analyser et extraire ·	
	2.4 Exercice complet — quotes.toscrape.com ·	
	2.5 Stocker en CSV et PostgreSQL	
03	 APIs REST — collecter des données en JSON	8-12
	3.1 Qu'est-ce qu'une API REST ·	
	3.2 Open-Meteo — météo sans clé ·	
	3.3 REST Countries — données pays ·	
	3.4 Frankfurter — taux de change BCE ·	
	3.5 Combiner plusieurs APIs	
04	 Reporting automatisé	13-17
	4.1 Architecture du script ·	
	4.2 Croiser NovaBrand avec les APIs ·	
	4.3 Générer un CSV enrichi ·	
	4.4 Générer un rapport texte ·	
	4.5 Exporter vers PostgreSQL	
05	Checklist et récapitulatif	18

Contexte du projet – DataPulse

1.1 Ce que l'on va construire

NovaBrand veut enrichir ses données de campagnes avec des sources externes : météo au moment des campagnes, indicateurs économiques des pays cibles, et taux de change pour convertir les revenus. Ce projet vous apprend à collecter ces données automatiquement depuis le web et des APIs publiques, à les croiser avec le dataset NovaBrand, et à générer un rapport hebdomadaire automatisé.

PIPELINE DATAPULSE

Demi-journée 1

- ▶ Scraping — `quotes.toscrape.com`
- ▶ `requests` + `BeautifulSoup`
- ▶ Extraction : texte, auteur, tags
- ▶ Export CSV + PostgreSQL

Demi-journée 2

- ▶ API Open-Meteo — météo
- ▶ API REST Countries — pays
- ▶ API Frankfurter — taux change
- ▶ Stocker en DataFrames

Demi-journée 3

- ▶ Croiser NovaBrand + APIs
- ▶ Générer rapport enrichi CSV
- ▶ Générer rapport texte manager
- ▶ Export PostgreSQL

1.2 Le web comme source de données

Une grande partie des données utiles en marketing se trouve sur des pages web publiques. Le scraping consiste à extraire automatiquement ces données depuis le HTML d'une page. Pour ce workshop, on utilise `quotes.toscrape.com` — un site conçu expressément pour apprendre le scraping, sans restriction.

1.3 Les APIs : pourquoi et comment

Une **API REST** retourne directement du **JSON structuré**. Les trois APIs utilisées sont publiques, gratuites, et ne nécessitent aucune clé.

API	URL de base	Ce qu'elle fournit	Clé
Open-Meteo	<code>api.open-meteo.com/v1</code>	Météo actuelle et historique	×
REST Countries	<code>restcountries.com/v3.1</code>	Population, langues, monnaies	×
Frankfurter	<code>api.frankfurter.dev/v2</code>	Taux de change BCE	×

Ce projet utilise `campaigns_clean.csv` du Projet 1. Vérifiez qu'il est disponible avant de commencer.

Scraping web – requests et BeautifulSoup

2.1 Comment fonctionne une page web

Quand vous ouvrez un site dans votre navigateur, il télécharge un fichier HTML — un texte structuré avec des balises. Le scraping consiste à télécharger ce même HTML avec Python, puis à en extraire les informations. Pour voir le HTML d'une page : clic droit → **Inspecter**.

```
$ Terminal PowerShell – dans le dossier datapulse/

cd C:\\Users\\VotreNom\\Documents
  create datapulse
  cd datapulse
  python -m venv .env
  .env\\Scripts\\Activate.ps1
  pip install requests beautifulsoup4 pandas sqlalchemy psycopg2-binary
```

2.2 requests — télécharger le HTML d'une page

```
scraping_intro.py

import requests

url = "http://quotes.toscrape.com/"
response = requests.get(url)

# 200 = succès, 404 = introuvable, 403 = accès refusé
print(response.status_code)
print(response.text[:500]) # les 500 premiers chars du HTML
```

📌 CODES HTTP

200 = succès · **403** = interdit · **404** = introuvable · **500** = erreur serveur. Toujours vérifier que le code est 200 avant de parser.

2.3 BeautifulSoup — analyser le HTML

```
scraping_bs4.py

from bs4 import BeautifulSoup

soup = BeautifulSoup(response.text, "html.parser")

print(soup.title.text) # "Quotes to Scrape"
quotes = soup.find_all("div", class_="quote")
print(f"{{len(quotes)}} citations sur cette page") # 10
```

```
premier = quotes[0]
print(premier.find("span", class_="text").text)
print(premier.find("small", class_="author").text)
```

2.4 Exercice complet — scraper quotes.toscrape.com

Le site contient 10 pages de citations. On scrape les **10 pages en entier** et on collecte toutes les citations.

```
scraping.py

import requests
import pandas as pd
from bs4 import BeautifulSoup
import time

base_url = "http://quotes.toscrape.com"
all_quotes = []
page = 1

while True:
    url = f"{base_url}/page/{page}/"
    response = requests.get(url)
    if response.status_code != 200:
        break

    soup = BeautifulSoup(response.text, "html.parser")
    quote_divs = soup.find_all("div", class_="quote")
    if not quote_divs:
        break

    for div in quote_divs:
        texte = div.find("span", class_="text").text.strip()
        auteur = div.find("small", class_="author").text.strip()
        tags = [t.text for t in div.find_all("a", class_="tag")]
        all_quotes.append({"page": page, "texte": texte,
                           "auteur": auteur, "tags": ", ".join(tags)})

    print(f"Page {page} : {len(quote_divs)} citations")
    page += 1
    time.sleep(0.5)

df_quotes = pd.DataFrame(all_quotes)
df_quotes.to_csv("quotes.csv", index=False)
print(f"✅ {len(df_quotes)} citations → quotes.csv")
```

💡 TIME.SLEEP()

Attendre 0.5 secondes entre chaque page est une bonne pratique — cela évite de surcharger le serveur. Sur quotes.toscrape.com c'est inutile techniquement, mais c'est une habitude à prendre.

2.5 Stocker dans PostgreSQL

```
from sqlalchemy import create_engine
engine = create_engine("postgresql+psycopg2://postgres:@localhost:5432/campaigniq")
df_quotes.to_sql("quotes", con=engine, if_exists="replace", index=False)
```

```
print("Table 'quotes' chargée ✅")  
engine.dispose()
```


APIs REST – collecter des données en JSON

3.1 Qu'est-ce qu'une API REST

Une API REST expose des données structurées accessibles via une URL. On envoie une requête HTTP, le serveur répond avec du **JSON** – un format texte structuré que Python lit directement avec `.json()`.

ANATOMIE D'UNE REQUÊTE API

`https://api.frankfurter.dev/v2/rates?base=EUR&symbols=USD,GBP,JPY`

DOMAINE

`api.frankfurter.dev`

ENDPOINT

`/v2/rates`

PARAMÈTRE 1

`base=EUR`

PARAMÈTRE 2

`symbols=USD,GBP,JPY`

Les paramètres commencent après `?` et sont séparés par `&`

```
import requests
response = requests.get("https://api.frankfurter.dev/v2/rates?base=EUR&symbols=USD,GBP")
data = response.json()
print(data)
# {'base': 'EUR', 'date': '2025-01-15', 'rates': {'USD': 1.032, 'GBP': 0.843}}
print(data["rates"]["USD"]) # 1.032
```

3.2 Open-Meteo – données météo sans clé API

📌 COORDONNÉES DES CAPITALES NOVABRAND

Paris (48.8566, 2.3522) · Berlin (52.52, 13.405) · Madrid (40.4168, -3.7038) · Rome (41.9028, 12.4964) · Amsterdam (52.3676, 4.9041)

```
api_meteo.py

import requests
import pandas as pd

capitales = {
    "France" : (48.8566, 2.3522),
    "Germany" : (52.5200, 13.4050),
    "Spain" : (40.4168, -3.7038),
    "Italy" : (41.9028, 12.4964),
    "Netherlands": (52.3676, 4.9041),
}

rows = []
```

```

for pays, (lat, lon) in capitales.items():
    url = (
        f"https://api.open-meteo.com/v1/forecast"
        f"?latitude={{lat}}&longitude={{lon}}"
        f"&current=temperature_2m,precipitation"
    )
    r = requests.get(url).json()
    rows.append({{
        "country"      : pays,
        "temperature_c" : r["current"]["temperature_2m"],
        "precipitation_mm": r["current"]["precipitation"],
    }})
    print(f"  {{pays}} : {{r['current']['temperature_2m']}}°C")

df_meteo = pd.DataFrame(rows)
df_meteo.to_csv("meteo.csv", index=False)

```

3.3 REST Countries — données sur les pays

```
api_countries.py

import requests, pandas as pd

pays_novabrand = ["France", "Germany", "Spain", "Italy", "Netherlands"]
rows = []

for pays in pays_novabrand:
    url = (f"https://restcountries.com/v3.1/name/{pays}"
          f"?fields=name,population,area,currencies,languages")
    data = requests.get(url).json()[0]
    rows.append({
        "country" : pays,
        "population": data["population"],
        "area_km2" : data["area"],
        "currency" : ", ".join(data["currencies"].keys()),
        "languages" : ", ".join(data["languages"].values()),
    })

df_countries = pd.DataFrame(rows)
df_countries.to_csv("countries.csv", index=False)
print(df_countries)
```

⚠ JSON IMBRIQUÉ

`currencies` et `languages` sont des dicts imbriqués. On utilise `.keys()` et `.values()` pour extraire les données. Toujours explorer la structure avec `print(data)` avant d'écrire le code d'extraction.

3.4 Frankfurter — taux de change BCE

```
api_taux.py

import requests, pandas as pd

url = "https://api.frankfurter.dev/v2/rates?base=EUR&symbols=USD,GBP,CHF,JPY"
data = requests.get(url).json()
print(f"Date des taux : {data['date']}")
print(data["rates"])

df_taux = pd.DataFrame([{"date": data["date"], **data["rates"]}])
df_taux.to_csv("taux_change.csv", index=False)

# Taux historique — comparaison sur 1 an
url_h = "https://api.frankfurter.dev/v2/rates?base=EUR&symbols=USD&date=2024-01-15"
data_h = requests.get(url_h).json()
taux_actuel = data["rates"]["USD"]
taux_ancien = data_h["rates"]["USD"]
```

```
variation = ((taux_actuel - taux_ancien) / taux_ancien) * 100  
print(f"Variation EUR/USD sur 1 an : {{variation:+.2f}}%")
```

3.5 Combiner plusieurs APIs — données enrichies par pays

```
enrichissement_pays.py

import pandas as pd

df_meteo = pd.read_csv("meteo.csv")
df_countries = pd.read_csv("countries.csv")
df_taux = pd.read_csv("taux_change.csv")

# Jointure météo + pays sur la colonne "country"
df_ref = df_meteo.merge(df_countries, on="country", how="left")

# Ajouter le taux EUR/USD (même valeur pour tous les pays)
df_ref["eur_usd_rate"] = df_taux["USD"].iloc[0]
df_ref["pop_density"] = (df_ref["population"] / df_ref["area_km2"]).round(1)

print(df_ref[["country", "temperature_c", "population", "eur_usd_rate"]])
df_ref.to_csv("pays_reference.csv", index=False)
print("✅ pays_reference.csv créé")
```

💡 MERGE() — LA JOINTURE PANDAS

`df.merge(autre_df, on='colonne', how='left')` fonctionne comme un `LEFT JOIN SQL` : on garde toutes les lignes du premier DataFrame et on ajoute les colonnes du second quand les valeurs correspondent.

3.6 Gérer les erreurs d'API — try/except

```
def get_api(url, max_retries=3):
    import time
    for tentative in range(max_retries):
        try:
            r = requests.get(url, timeout=10)
            r.raise_for_status()
            return r.json()
        except requests.exceptions.Timeout:
            print(f"Timeout - tentative {tentative+1}/{max_retries}")
            time.sleep(2)
        except requests.exceptions.HTTPError as e:
            print(f"Erreur HTTP {e.response.status_code}")
            return None
    return None
```

Reporting automatisé

4.1 Architecture du script de reporting

CE QUE LE SCRIPT VA PRODUIRE

Entrées

- `campaigns_clean.csv`
- API Open-Meteo
- API REST Countries
- API Frankfurter

Traitement

- Charger les données
- Appeler les 3 APIs
- Merger sur "country"
- Calculer KPIs EUR + USD

Sorties

- `rapport_enrichi.csv`
- `rapport_semaine.txt`
- PostgreSQL table `rapport_hebdo`

4.2 Croiser NovaBrand avec les données API

reporting.py – partie 1 : chargement et enrichissement

```
import pandas as pd
import requests
from datetime import date
from sqlalchemy import create_engine

# 1. Charger le dataset NovaBrand
df = pd.read_csv("campaigns_clean.csv")
df["date"] = pd.to_datetime(df["date"])

# 2. Agréger par pays – 7 derniers jours
date_limite = df["date"].max() - pd.Timedelta(days=7)
df_s = df[df["date"] ≥ date_limite]
kpi_pays = df_s.groupby("country").agg(
    nb_campagnes = ("campaign_id", "count"),
    spend_eur = ("spend", "sum"),
    revenue_eur = ("revenue", "sum"),
    conversions = ("conversions", "sum"),
    roas_moyen = ("roas", "mean"),
).round(2).reset_index()

# 3. Taux de change EUR/USD
taux_data = requests.get(
    "https://api.frankfurter.dev/v2/rates?base=EUR&symbols=USD"
).json()
taux_usd = taux_data["rates"]["USD"]
kpi_pays["spend_usd"] = (kpi_pays["spend_eur"] * taux_usd).round(2)
kpi_pays["revenue_usd"] = (kpi_pays["revenue_eur"] * taux_usd).round(2)

# 4. Météo actuelle par capitale
capitales = {
    "France": (48.8566, 2.3522), "Germany": (52.52, 13.405),
```

```

    "Spain": (40.4168, -3.7038), "Italy": (41.9028, 12.4964),
    "Netherlands": (52.3676, 4.9041),
}}
meteo_rows = []
for pays, (lat, lon) in capitales.items():
    r = requests.get(
        f"https://api.open-meteo.com/v1/forecast?latitude={{lat}}&longitude={{lon}}&current=temperature_2m,precipitation"
    ).json()
    meteo_rows.append({"country": pays,
                       "temperature_c": r["current"]["temperature_2m"],
                       "precipitation_mm": r["current"]["precipitation"]})
df_meteo = pd.DataFrame(meteo_rows)

# 5. Merger tout ensemble
rapport = kpi_pays.merge(df_meteo, on="country", how="left")
rapport["taux_eur_usd"] = taux_usd
rapport["date_rapport"] = str(date.today())
rapport = rapport.sort_values("revenue_eur", ascending=False)
print(rapport)

```

4.3 Générer le CSV enrichi

```
rapport.to_csv("rapport_enrichi.csv", index=False)
print(f"✅ rapport_enrichi.csv ({len(rapport)} lignes)")
```

4.4 Générer un rapport texte lisible

Un CSV est utile pour d'autres analyses. Mais un manager veut quelque chose qu'il peut lire en 30 secondes. On génère un fichier texte formaté.

```
reporting.py - partie 2 : rapport texte

from datetime import date
aujourd_hui = date.today().strftime("%d/%m/%Y")

lignes = [
    f"=====",
    f"  RAPPORT NOVABRAND - {aujourd_hui}",
    f"=====",
    "",
    f"  Taux EUR/USD : {{taux_usd}} (BCE)",
    f"  Campagnes   : {{len(df_s)}}",
    "",
    f"  {{'PAYS':<14}} {{'SPEND €':>9}} {{'REVENUE €':>11}} {{'ROAS':>6}} {{'T°':>5}}",
    f"  {{'-'*48}}",
]

for _, row in rapport.iterrows():
    lignes.append(
        f"    {{row['country']:<14}} {{row['spend_eur']:>9,.0f}}"
        f"    {{row['revenue_eur']:>11,.0f}} {{row['roas_moyen']:>6.2f}}"
        f"    {{row['temperature_c']:>4.1f}}°C"
    )

lignes += [
    f"  {{'-'*48}}",
    f"  TOTAL SPEND   : {{rapport['spend_eur'].sum():.0f}} EUR",
    f"  TOTAL REVENUE : {{rapport['revenue_eur'].sum():.0f}} EUR",
    f"  ROAS GLOBAL   : {{rapport['revenue_eur'].sum()/rapport['spend_eur'].sum():.2f}}",
]

rapport_texte = "\n".join(lignes)
with open("rapport_semaine.txt", "w", encoding="utf-8") as f:
    f.write(rapport_texte)
print(f"✅ rapport_semaine.txt généré")
```

4.5 Exporter vers PostgreSQL

```
engine = create_engine("postgresql+psycopg2://postgres:@localhost:5432/campaigniq")
rapport.to_sql("rapport_hebdo", con=engine, if_exists="replace", index=False)
print(f"✅ Table rapport_hebdo chargée")
engine.dispose()
```


AUTOMATISER CE SCRIPT

Sur Windows, le **Planificateur de tâches** permet de lancer un script Python automatiquement chaque semaine. Le script tourne sans intervention humaine et produit ses fichiers automatiquement.

Checklist et récapitulatif

Bilan des compétences — DataPulse

Compétence	Commande clé	✓
Télécharger le HTML d'une page web	<code>requests.get(url)</code> <code>.status_code == 200</code>	
Parser du HTML avec BeautifulSoup	<code>BeautifulSoup(html, "html.parser")</code> <code>.find()</code> · <code>.find_all()</code>	
Extraire texte et attributs CSS	<code>.text.strip()</code> · <code>class_="nom"</code>	
Scraper plusieurs pages (pagination)	Boucle <code>while True</code> avec incrémentation de page	
Appeler une API REST et parser le JSON	<code>response.json()</code> · <code>data["key"]</code>	
Passer des paramètres dans l'URL	<code>?base=EUR&symbols=USD</code>	
Extraire du JSON imbriqué	<code>data["currencies"].keys()</code>	
Gérer les erreurs d'API	<code>raise_for_status()</code> · <code>timeout=10</code>	
Combiner plusieurs DataFrames	<code>df.merge(autre, on="country", how="left")</code>	
Convertir des montants avec un taux	<code>df["col"] * taux_usd</code>	
Générer un fichier texte formaté	<code>open("fichier.txt", "w")</code> · f-strings	
Charger le rapport dans PostgreSQL	<code>df.to_sql("rapport_hebdo", engine)</code>	

Récapitulatif — ce que vous avez construit

CE QUE VOUS AVEZ CONSTRUIT

scraping.py

- `requests.get()`
- `BeautifulSoup()`
- 10 pages · 100 citations

`quotes.csv`

api_*.py

- Open-Meteo API
- REST Countries API
- Frankfurter API

`pays_reference.csv`

reporting.py

- Charge NovaBrand
- Appelle les 3 APIs
- Merge + calcul KPIs
- CSV + TXT + PostgreSQL

💡 CE QUI VOUS ATTEND AU PROJET 3

Le Projet 3 (RetailLens) réutilise la base PostgreSQL `campaigniq` et y ajoute 500 000 lignes de transactions e-commerce réelles. On y applique des requêtes SQL avancées (CTEs, window

functions) puis on visualise tout depuis Power BI Desktop connecté directement à PostgreSQL.